

433-HW5

Alex Caramanico

October 5, 2022

Problem 1

```
fixedpoint = function(g, x0, tol=1e-8) {
  cat('Fixed Point Algorithm: \n')
  xn = x0 # set initial values
  rootcheck = abs(g(xn)-xn) # check closeness of functional value to 0
  ii = 1 # count iterations
  cat('At step', ii, ', x is', xn, '\n')
  while (rootcheck > tol && ii < 100) {
    xn = g(xn) # update xn based on fixed point algorithm
    rootcheck = abs(g(xn)-xn)
    ii = ii+1
    cat('At step', ii, ', x is', xn, '\n')
  }
}

newt_raph = function(f, fp, x0, tol=1e-8) {
  cat('Newton-Raphson Method: \n')
  xn = x0 # initial values
  fval = f(xn)
  root_check = abs(fval)
  ii = 1 # number of iterations
  cat('At step', ii, ', x is', xn, '\n')
  while (root_check > tol) { # loop until f(x) is close enough to 0
    xn = xn - f(xn)/fp(xn) # move to next x value
    fval = f(xn) # update functional value
    root_check = abs(fval) # update root checker
    ii = ii + 1
    cat('At step', ii, ', x is', xn, '\n')
  }
}

func = function(x) {cos(x)} # f(x)
gunc = function(x) {cos(x)-x} # g(x)
gunc_p = function(x) {-sin(x)-1} # g'(x)

system.time(fixedpoint(func,0))[3] # run functions, display time taken
```

```
## Fixed Point Algorithm:
## At step 1 , x is 0
```

```

## At step 2 , x is 1
## At step 3 , x is 0.5403023
## At step 4 , x is 0.8575532
## At step 5 , x is 0.6542898
## At step 6 , x is 0.7934804
## At step 7 , x is 0.7013688
## At step 8 , x is 0.7639597
## At step 9 , x is 0.7221024
## At step 10 , x is 0.7504178
## At step 11 , x is 0.731404
## At step 12 , x is 0.7442374
## At step 13 , x is 0.7356047
## At step 14 , x is 0.7414251
## At step 15 , x is 0.7375069
## At step 16 , x is 0.7401473
## At step 17 , x is 0.7383692
## At step 18 , x is 0.7395672
## At step 19 , x is 0.7387603
## At step 20 , x is 0.7393039
## At step 21 , x is 0.7389378
## At step 22 , x is 0.7391844
## At step 23 , x is 0.7390183
## At step 24 , x is 0.7391302
## At step 25 , x is 0.7390548
## At step 26 , x is 0.7391056
## At step 27 , x is 0.7390714
## At step 28 , x is 0.7390944
## At step 29 , x is 0.7390789
## At step 30 , x is 0.7390893
## At step 31 , x is 0.7390823
## At step 32 , x is 0.739087
## At step 33 , x is 0.7390838
## At step 34 , x is 0.739086
## At step 35 , x is 0.7390845
## At step 36 , x is 0.7390855
## At step 37 , x is 0.7390849
## At step 38 , x is 0.7390853
## At step 39 , x is 0.739085
## At step 40 , x is 0.7390852
## At step 41 , x is 0.7390851
## At step 42 , x is 0.7390852
## At step 43 , x is 0.7390851
## At step 44 , x is 0.7390851
## At step 45 , x is 0.7390851
## At step 46 , x is 0.7390851
## At step 47 , x is 0.7390851

```

```

## elapsed
## 0.01

```

```

system.time(newt_raph(gunc,gunc_p,0))[3]

```

```

## Newton-Raphson Method:

```

```
## At step 1 , x is 0
## At step 2 , x is 1
## At step 3 , x is 0.7503639
## At step 4 , x is 0.7391129
## At step 5 , x is 0.7390851
```

```
## elapsed
##      0
```

*# The Newton-Raphson Method is significantly faster at the same starting value.
Fixed Point Algorithm seems to get closer to the root less with each step,
that is the value of x changes by less and less as the algorithm converges.*

Problem 2

```
halley = function(f, fprime, f2prime, x0, tol = 1e-8) {
  cat('Original Halley Method \n')
  cat('The initial value is', x0, '\n')
  xn = x0
  fval = f(xn)
  root_check = abs(fval) # check if value of function is close enough to 0
  ii = 1 # iterations
  while (root_check > tol) {
    numerator = f(xn) # variable for numerator of algorithm
    denom1 = fprime(xn) # variables for denominator components
    denom2 = (f(xn)*f2prime(xn))/(2*fprime(xn))
    denominator = denom1 - denom2
    xn = xn - (numerator/denominator) # algorithm for next value
    cat('At step', ii, ', x is', xn, '\n')
    root_check = f(xn) # update root checker
    ii = ii + 1 # increment iterations
  }
  cat('\n') # extra line for readability
}
```

```
halley_alt = function(f, x0, m, k, tol=1e-8) {
  cat('Alternate Halley Method \n')
  cat('The initial value is', x0, '\n')
  xn = x0
  funcval = f(xn)
  check_root = abs(f(xn)) # check whether value is close enough to 0
  ii = 1
  while (check_root > tol) {
    num = (m-1)*xn^m+(m+1)*k # numerator
    denom = (m+1)*xn^m + (m-1)*k # denominator
    xn = xn*(num/denom) # update xn
    cat('At step', ii, ', x is', xn, '\n')
    check_root = f(xn)
    ii = ii + 1
  }
}
```

```

func = function(x) {exp(x-1)-0.5} # f(x)
funcprime = function(x) {exp(x-1)} # f'(x)
func2prime = function(x) {exp(x-1)} # as formality, f''(x) = f'(x)
alt_func = function(x) {x^3-4} # function in form f(x) = x^m-k

halley(func,funcprime,func2prime, 1)

```

```

## Original Halley Method
## The initial value is 1
## At step 1 , x is 0.3333333
## At step 2 , x is 0.3068544
## At step 3 , x is 0.3068528

```

```

halley_alt(alt_func,6,3,4) # both rely on good initial guess

```

```

## Alternate Halley Method
## The initial value is 6
## At step 1 , x is 3.082569
## At step 2 , x is 1.836821
## At step 3 , x is 1.590642
## At step 4 , x is 1.587401
## At step 5 , x is 1.587401

```

Problem 3

```

riemann_1 = function(f,a,b,n) { # f(x), lower and upper bound, n sub-intervals
  h = (b-a)/n # length of each sub-interval
  xi = seq(a,b-h,by=h) # create sequence for each x point
  # stop at b-h for left point sum, start at a+h for right point sum
  # f(x) is strictly decreasing on the interval, so
  # left point sum is an overestimate, right point is an underestimate
  f_xi = f(xi) # sequence of functional values at each x
  riemann_area = h*sum(f_xi)
  return(riemann_area)
}

trap = function(f,a,b,n) {
  h = (b-a)/n
  xi = seq(a,b,by=h) # sequence for each x point
  f_xi = f(xi) # sequence of functional values at each x
  trap_area = h/2*(f_xi[1]+sum(2*f_xi[2:n])+f_xi[n+1]) # trapezoidal algorithm
  return(trap_area)
}

func = function(x) {exp(-0.5*x^2)}

cat('n = 10 \n')

```

```

## n = 10

```

```
cat('Riemann:', riemann_l(func,0,1,10),'\n') # actual value: ~0.855624
```

```
## Riemann: 0.8747922
```

```
cat('Trapezoidal:', trap(func,0,1,10), '\n')
```

```
## Trapezoidal: 0.8551188
```

```
# Riemann approximation is less accurate than the trapezoidal approximation.  
# Difference between the two depends on whether Riemann uses right or left points,  
# right point Riemann is less than the trapezoidal approximation, while left point  
# Riemann is greater than the trapezoidal approximation.
```

```
cat('n = 100 \n')
```

```
## n = 100
```

```
cat('Riemann:', riemann_l(func,0,1,100),'\n')
```

```
## Riemann: 0.8575867
```

```
cat('Trapezoidal:', trap(func,0,1,100), '\n')
```

```
## Trapezoidal: 0.8556193
```

```
cat('n = 10000 \n')
```

```
## n = 10000
```

```
cat('Riemann:', riemann_l(func,0,1,10000),'\n')
```

```
## Riemann: 0.8556441
```

```
cat('Trapezoidal:', trap(func,0,1,10000), '\n')
```

```
## Trapezoidal: 0.8556244
```

```
cat('n = 1 million \n')
```

```
## n = 1 million
```

```
cat('Riemann:', riemann_l(func,0,1,1000000),'\n')
```

```
## Riemann: 0.8556246
```

```
cat('Trapezoidal:', trap(func,0,1,1000000), '\n')
```

```
## Trapezoidal: 0.8556244
```

```
# As the number of sub-intervals increases, both algorithms converge,  
# albeit at different "speeds".
```